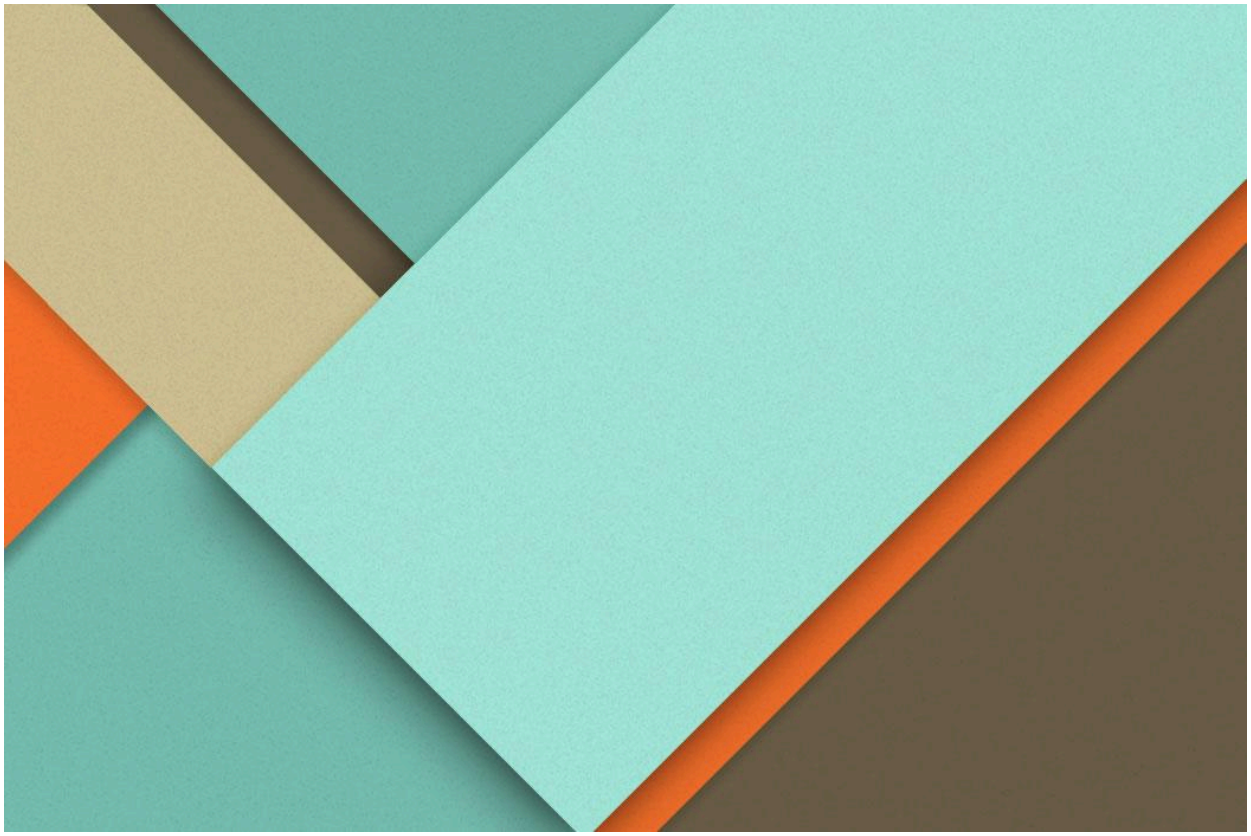


LLM Orchestrator for CI

4041

فاطمه اسماعیلی
هما سادات حسینی شکوه
پارمیس وظیفه

Overview

هدف ایجاد یک عامل مبتنی بر LLM به نام Meta Mind است. تا بتواند نام یک مسئله را از کاربر گرفته و در ادامه روش حل بهینه این سوال را مشخص و مقدار پارامتر های پیشنهادی را پیشنهاد بدهد.

Goals

1. ساخت یک عامل هوشمند مبتنی بر LLM (MetaMind)
2. End-to-End Orchestration
3. پیاده سازی به گونه ای که همه الگوریتم ها از یک API مشترک استفاده کنند
4. حل ۴ مسئله به عنوان بنچمارک TSP (مسیریابی), Function Optimization, Titanic Classificaton, Clustering
5. ارائه مستندات و گزارش ها

Specifications

main.py

در این مرحله از کاربر نام مسئله را میگیریم و در ادامه یک prompt اماده میکنیم در ادامه LLMOrchestrator.analyze_problem نوع مسئله و روش و پارامتر هارا پیشنهاد میکند و به کمک آن object problems و method object را میسازد.

در ادامه با interpret_results از LLM نتایج را می گیرد.

agent.py

در این بخش با OpenRouter مدل API را صدا میزنیم. خروجی یک فایل JSON شامل problem_type selected_method, reasoning, parameters را میدهد. خروجی مدل یک feedback result مربوط به متریک های پروژه است

(متریک ها به ترتیب : TSP: طول مسیر / best distance / (اختیاری gap)

Function Optimization: best fitness / convergence

Classification: Accuracy/Precision/Recall/F1/AUC

(Clustering: Silhouette/DB/CH

base.py

در این بخش یک API یکسان برای همه متد ها تعریف میشود شامل fit, predict و validate_parameters

evolutionary.py

در این بخش الگوریتم های تکاملی وGA/GP تعریف شده اند

GA: جمعیت اولیه رندوم initialize میشود و انتخاب بر اساس تورنمنت است و مرتب‌سازی براساس fitness و انتخاب elitism هر دو روش crossover و mutation استفاده میشود و

برای مسائل ترکیبی یا مسائلی که راه‌حلشان به شکل ساختار/لیست/پرمیوتیشن است (مثلاً TSP).

```
defaults = {
            "population_size": 100,
            "generations": 500,
            "crossover_rate": 0.8,
            "mutation_rate": 0.1,
            "selection": "tournament",
            "tournament_size": 3,
            "elitism": 2
        }
```

PSO: برای بهینه‌سازی پیوسته (بردار عددی)، مثل Ackley/Rastrigin/Rosenbrock.مجموعه‌ای از ذرات را در فضای جواب حرکت می‌دهد؛ هر ذره دارای موقعیت و سرعت است و با تابع problem.evaluate ارزیابی می‌شود. در هر تکرار، سرعت ذرات با اینرسی (w)، تمایل به بهترین موقعیت شخصی (c1) و تمایل به بهترین موقعیت سراسری (c2) به روز می‌شود و سپس موقعیت‌ها به محدوده‌ی problem.bounds محدود می‌شوند. PSO بهترین جواب سراسری (global best) را دنبال می‌کند و مقدار بهترین fitness را در هر iteration در convergence_history ثبت می‌کند؛ خروجی نهایی، همان بهترین بردار پارامترها (بهترین موقعیت) است که کمترین مقدار تابع هدف را داده است.

```
defaults = {
            "n_particles": 50,
            "max_iterations": 500,
            "w": 0.7,    # Inertia weight
            "c1": 1.5,   # Cognitive coefficient
```

```
            "c2": 1.5,  # Social coefficient
            "v_max": 0.5
        }
```

ACO:برای مسائل مسیر/گراف مثل TSP استفاده می‌شود. یک ماتریس فرومون اولیه ساخته می‌شود و در هر iteration، هر مورچه با استفاده از احتمال‌های مبتنی بر فرومون (alpha) و یک heuristic مثل (beta) می‌سازد. طول/کیفیت هر تور با problem.evaluate سنجیده می‌شود، سپس فرومون‌ها با نرخ تبخیر کاهش می‌یابند و به مسیرهای بهتر به اندازه‌ی Q / distance فرومون اضافه می‌شود تا در تکرارهای بعدی احتمال انتخاب آن یال‌ها بیشتر شود. بهترین مسیر در طول اجرا نگه‌داری می‌شود و روند بهترین فاصله در convergence_history ذخیره می‌گردد

```
defaults = {
            "n_ants": 30,
            "max_iterations": 200,
            "alpha": 1.0,          # Pheromone importance
            "beta": 2.0,           # Heuristic importance
            "evaporation_rate": 0.5,
            "Q": 100               # Pheromone deposit factor
        }
```

GeneticProgramming (GP): فایل برای تولید شکل درخت پیاده‌سازی شده است هر فرد یک درخت شامل عملگرها (مثل +، -، *، /) و ترمینال‌ها (مثل x یا ثابت‌ها) است که با کلاس Node ساخته می‌شود و با یک evaluator روی داده‌ها محاسبه می‌گردد. جمعیت اولیه با درخت‌های تصادفی ایجاد می‌شود و کیفیت هر درخت توسط problem.evaluate(tree, evaluator) تعیین می‌گرد. در هر نسل بهترین درخت انتخاب و نگه‌داری می‌شود و روند بهترین مقدار ثبت می‌شود خروجی نهایی هم همان بهترین درخت است که بهترین fitness را ایجاد کرده است.

```
defaults = {
            "population_size": 200,
            "generations": 50,
            "max_depth": 6,
            "crossover_rate": 0.9,
            "mutation_rate": 0.1,
            "function_set": ["+", "-", "*", "/"],
```

```
            "terminal_set": ["x", "constants"],
            "parsimony_coefficient": 0.001
        }
```

neural.py

در این بخش روش های شبکه عصبی پیاده سازی میشوند:

Perceptron: یک مدل خطی ساده است که وزن‌ها و بایاس را به‌صورت دستی یاد می‌گیرد؛ در مرحله‌ی fit وزن‌ها با قانون یادگیری پرسپترون و بر اساس خطای پیش‌بینی به‌روزرسانی می‌شوند. پارامتر های دیفالت به فرم زیر است.

```
default_params = {
            "learning_rate": 0.01,
            "max_epochs": 100,
            "bias": True
        }
```

MLP: یک شبکه‌ی عصبی چندلایه است که به‌صورت یک wrapper روی sklearn.neural_network.MLPClassifier ساخته شده تا آموزش و پیش‌بینی سریعتر و پایدارتر داشته باشیم. مدل با backpropagation آموزش می‌بیند سپس برای پایش کیفیت یادگیری، مقدار loss هر epoch از loss_curve_ گرفته می‌شود و به‌عنوان convergence_history ذخیره می‌گردد. خروجی predict هم همان پیش‌بینی برچسب توسط شبکه است.

```
defaults = {
            "hidden_layers": [64, 32],
            "activation": "relu",
            "learning_rate": 0.001,
            "max_epochs": 500,
            "optimizer": "adam"
        }
```

SOM: یک مدل بدون ناظر برای نگاشت داده‌های چندبعدی روی یک شبکه‌ی دوبعدی از نورون‌هاست که با حفظ ساختار همسایگی، داده‌ها را خودسازمان‌دهی می‌کند. در آموزش (fit) برای هر نمونه نزدیک‌ترین نورون (BMU) پیدا می‌شود و سپس وزن‌های BMU و همسایه‌هایش با نرخ یادگیری و شعاع همسایگی به سمت نمونه حرکت داده می‌شوند تا نقشه به‌تدریج شکل بگیرد.

```
defaults = {
```

```
            "map_size": (10, 10),
            "learning_rate_initial": 0.5,
            "max_epochs": 1000,
            "topology": "rectangular"
        }
```

HopfieldNetwork: شبکه هاپفیلد یک associative memory است که الگوها را در شبکه‌ی بازگشتی ذخیره می‌کند. در fit وزن‌ها معمولاً با جمع outer product الگوهای آموزشی ساخته می‌شوند تا شبکه بتواند آن‌ها را به خاطر بسپارد. در مرحله recall از یک ورودی ناقص شروع می‌کند و با به‌روزرسانی تکراری نورون‌ها وضعیت شبکه را به سمت نزدیک‌ترین الگوی ذخیره‌شده همگرا می‌کند خروجی نهایی الگوی بازیابی‌شده از حافظه است.

```
defaults = {
            "max_iterations": 100,
            "async_update": True
        }
```

fuzzy.py

یک کنترل‌کننده‌ی فازی ساده است با fuzzy set های low/medium/high با membership مثلثی و روش های defuzzification توسط یک centroid

```
defaults = {
            "n_membership_functions": 3,
            "membership_type": "triangular",
            "defuzzification": "centroid",
            "rules": "standard"
        }
```

tsp.py

مسئله TSP را مدل می‌کند: ورودی مختصات شهرها dist_matrix است. پاسخ نهایی به‌شکل یک permutation از شهرها تولید می‌شوند (initialize_random_solution) و تابع evaluate(route) طول مسیر را با احتساب برگشت به شهر شروع محاسبه می‌کند. برای پشتیبانی از GA، متدهای crossover (تولید فرزندهای معتبر بدون تکرار شهر) و mutate (swap دو

شهر با احتمال مشخص) را دارد، و apply_2opt هم پیاده‌سازی کرده که با برعکس‌کردن بازه‌هایی از مسیر، تلاش می‌کند مسیر کوتاه‌تر پیدا کند.

optimization.py

فایل برای بهینه‌سازی توابع معیار پیوسته است . کلاس با توجه به نام تابع bounds را تعیین می‌کند و با initialize_random_solution یک بردار تصادفی داخل همان بازه می‌سازد. هسته‌ی کار evaluate(x) است که مقدار تابع هدف را روی بردار x حساب می‌کند تا الگوریتم‌هایی مثل PSO و ... با همین bounds و evaluate می‌توانند مسئله را حل کنند.

classification.py

مسئله‌ی طبقه‌بندی دیتاست Titanic را آماده می‌کند در load_and_preprocess دیتاست خوانده می‌شود و preprocess انجام میشود. متد evaluate(y_true, y_pred, y_prob) هم معیارهای استاندارد طبقه‌بندی مثل Accuracy/Precision/Recall/F1 و Confusion Matrix را می‌دهد و اگر احتمال داشته باشیم AUC-ROC را هم حساب می‌کند.

clustering.py

مسئله‌ی خوشه‌بندی را برای iris و mall حالت synthetic که با make_blobs داده‌ی مصنوعی تولید می‌کند.
Preprocessانجام میشود متد evaluate(labels) کیفیت خوشه‌بندی را با شاخص‌های Davies–Bouldin، Silhouette و Calinski–Harabasz گزارش می‌دهد و اگر y_true موجود باشد، Adjusted Rand Index را هم اضافه می‌کند.